

The Alberta Buck - Printable Receipt (DRAFT v0.1)

A Plain-Text, Independently Re-Verifiable Record of an Identity-Bound BUCK Payment

Perry Kundert

2026-05-29



The cryptography of the Alberta Buck identity layer names an *identity point* $M \in \mathbb{G}_1$ and binds a transaction; the artifact a human actually holds is a *receipt*. This document specifies that receipt: a plain-text slip, sized for an 80mm thermal printer, that prints the plain-text Identity of both parties and the transaction details, and embeds – as a wrapped text payload – everything required to re-verify the payment by re-scanning the slip and re-running the wallet’s verifiers. The receipt closes two gaps the on-chain proofs leave open: it carries each party’s `canonical_identity_data` so a point M resolves to a human name (verified by recomputing $M = \text{keccak}(\text{data}) \cdot G$), and it states which on-chain facts (registry records, the **Transfer** / **Spent*** event) anchor the proof. Verification is two-tier: *offline*, re-running the proof algebra and the point→human bridge from the embedded payload alone; and *chain-anchored*, confirming the embedded records against a node. Five receipt kinds are specified – an EOA transfer from a public or a private counterparty, and the B1, A1, and A2 Note flavors – each reusing the verifiers already shipped (`approve_receipt_verify`, `receipt_verify`, `verifiable_decrypt`, and for the private-issuer A2 case the recipient-blinded `verifyIssuerReenc` binding). (Decryptability, Identity)

Contents

1	What a Receipt Is (and Is Not)	3
2	Two Gaps the Proofs Leave to the Receipt	3
2.1	The point -> human bridge	3
2.2	The trust anchor	3
3	The Two-Tier Verification Model	3
4	The AB-RCPT/1 Envelope	4
5	Per-Scenario Payloads	5
5.1	coa-pub – EOA transfer from a public counterparty	5
5.2	coa-priv – EOA transfer from a private counterparty	5
5.3	note-b1 / note-a1 – Note from a public issuer	5
5.4	note-a2 – Note from a private issuer	5
6	The Physical Artifact (ESC/POS, 80mm, 203dpi, 576px)	6
7	Verification Procedure	7
8	Wallet UX and Storage	7
9	Security Considerations	8
10	Phasing	8
11	Open Questions	9

1 What a Receipt Is (and Is Not)

A BUCK receipt is a **record of a completed payment**, generated by one party (here always the *recipient*, whose Private-Identity wallet is set up and who has received funds) and printable on a commodity thermal printer. It is the bilateral-disclosure half of the system's privacy promise made tangible: bulk chain-analytics cannot reconstruct who paid whom, but either party to a payment can produce a slip that names the counterparty in the clear and proves it.

- **It IS:** human-readable; names both parties' plain-text Identity (a display subset on paper, the full KYC datum in the payload); states amount, kind, time, and on-chain references; self-verifying from its own text payload; and extensible with optional informational line items.
- **It is NOT:** a bearer instrument (it redeems nothing – the value already moved, and any Note nullifier it references is already spent); a chain explorer; or a re-derivation of the KYC binding of M to a legal person (that rests on the issuer's Pointcheval-Sanders credential, assumed here). The optional line items are **not** covered by verification.

2 Two Gaps the Proofs Leave to the Receipt

2.1 The point $\rightarrow$ human bridge

Every shipped proof names an identity *point* M . A receipt must name a *person*. The link is the preimage:

Definition 1. A party's Identity datum is the canonical JSON *canonical_identity_data* (the KYC fields, sorted-key, no-whitespace – Identity Example). Its identity scalar is $m = \text{keccak}(\text{canonical}) \bmod \text{ORDER}$ and its identity point is $M = m \cdot G$.

The receipt therefore carries each party's *canonical_identity_data*, and a verifier recomputes $\text{keccak}(\text{data}) \cdot G$ and checks it equals the named M . That single check turns a curve point into "Alice Johnson, Alberta Identity Card, Alberta, Canada."

Requirement (data availability). The generating wallet must *hold* the counterparty's *canonical_identity_data*. For a **public** identity it is published (off-chain attestation). For a **private** identity it is disclosed bilaterally when the parties approve one another (or when a Note is delivered) – the same exchange that lets each verify $M = \text{keccak}(\cdot) \cdot G$ against the credential they can decrypt. A wallet that never received the preimage can still print the *point* and the issuer reference, but not the human name (see *Open Questions*).

2.2 The trust anchor

Offline math proves the receipt is *internally* sound; it cannot prove the embedded keys and events are the *real* on-chain ones. That is a chain read. Hence two tiers, both specified below.

3 The Two-Tier Verification Model

Tier	Needs	Confirms
1 – offline	the slip's text payload only	the proof algebra verifies; both M recompute from their <i>canonical_identity_data</i>
2 – anchored	tier 1 + an RPC node	the embedded <code>pk</code> / <code>E_addr</code> / <code>isPublicIdentity</code> equal the registry's for each

Tier 1 says *"these proofs are valid and name these humans"*; tier 2 says *"and those identities and that payment are the genuine on-chain ones."* Together they are the non-deniable receipt of *Decryptability*. Per the design decision, the payload is **offline-complete**: every value tier 1 needs is embedded, and tier 2's references ride alongside so anchoring is optional, not required, to read a receipt.

Property 2. (Receipt soundness.) *If tier 1 passes, the slip's proofs are valid for the embedded public inputs and each named M is the hash of the printed Identity datum. If tier 2 also passes, those inputs are the on-chain registry records and the payment is a mined event. No party can forge a passing slip naming a counterparty who did not transact (proof unforgeability + event immutability); a generator may decline to print a receipt, but cannot print a false one.*

4 The AB-RCPT/1 Envelope

The verified **core** is a deterministic map, serialized canonically (CBOR), and printed as **AB-RCPT/1**. followed by base64url of the bytes, wrapped to the print width. (Text-only by decision – no QR; the payload is literally plain-text, re-typable and OCR-able, and survives a scanner-less verifier.) The receipt id is `base32(sha256(core))` truncated to a short handle. The optional line items live *outside* the core (they are unverified) and are printed in their own section, never in the payload.

Shown as JSON for readability (the wire form is CBOR):

```
{
  "v": 1,
  "type": "eoa-pub" | "eoa-priv" | "note-b1" | "note-a1" | "note-a2",
  "chainid": 1,
  "contracts": { "registry": "0x..", "buck": "0x..", "notes": "0x.." },
  "payer": Party,           // the counterparty being named
  "payee": Party,           // the wallet owner (recipient)
  "txn": Txn,
  "proofs": Proofs          // type-specific (below)
}

Party {
  "addr": "0x..",
  "kind": "public" | "private",
  "identity": "<canonical_identity_data>", // preimage; keccak->m, M=m*G
  "M": ["0x..", "0x.."],                 // named identity point
  "pk": ["0x..", "0x.."],                 // registered _pk (tier-2 anchor)
  "E_addr": { "R":["..", ".."], "C":["..", ".."] } // registered ct (private; tier-2)
}

Txn {
  "kind": "eoa-transfer" | "note-spend",
  "value": 250000000,                     // BUCK base units (6 dp)
  "timestamp": 1779999011,                 // block time
  "event": "Transfer" | "Spent" | "SpentA" | "SpentB",
  "txhash": "0x..", "block": 1234567, "logindex": 3,
  // note-spend adds:
```

```
"mint_txhash": "0x..", "mint_block": 1234500, "nullifier": "0x.."
}
```

Proofs is exactly the witness the matching shipped verifier consumes, plus the **payee self-naming** proof present in every kind – a **verifiable_decrypt** of the payee’s *own* registered **E_addr** revealing **payee.M** (the wallet owner is Private-Identity, so they name themselves the same way they name a private counterparty).

5 Per-Scenario Payloads

The wallet owner is the **payee** throughout; only the **payer**-naming and the txn anchor differ. Each row names the verifier that re-runs tier 1.

type	payer	payer-naming proof (verifier)	txn anchor
ea-pub	EOA, public	none – payer.M recomputed from public identity; pk anchors addr	Transfer (fr
ea-priv	EOA, private	ApproveReceipt { E_for_spender , cp_proof , vd_proof } (approve_receipt_verify)	Transfer
note-b1	issuer, public	Receipt { opening , cms [], issuer_sig , nullifier , face } (receipt_verify)	Mint + Spent
note-a1	issuer, public	Receipt {...} (nullifier tag A) (receipt_verify)	Mint + Spent
note-a2	issuer, private	vd_issuer (decrypt E_iss-for-rec) + issuer_reenc binding + γ (verifyIssuerReenc + coupling)	Mint + Spent

5.1 ea-pub – EOA transfer from a public counterparty

Lightest: the payer’s **identity** is public, so **payer.M** is recomputed from it and no payer proof is needed; the payer **pk** ties the **Transfer.from** address to that public record (tier 2). Payload = payer Party (public) + payee Party + payee **vd_proof** + the **Transfer** anchor.

5.2 ea-priv – EOA transfer from a private counterparty

The **ApproveReceipt** half: at approve time the payer re-encrypted their **M** for the payee (**E_for_spender**) and proved it (**cp_proof** = **verifyApprove**); the payee verifiably decrypts it to **payer.M** (**vd_proof**). Payload = both Parties (private) + **ApproveReceipt** proofs + payee self **vd_proof** + **Transfer** anchor + registry **pk**≠**E_addr** for both (tier 2).

5.3 note-b1 / note-a1 – Note from a public issuer

The **Receipt** four-link chain: the opening recomputes **cm**, **cm** is in the mint **cms**[], the issuer’s batch Schnorr binds the public issuer to it, and the nullifier was burned in the **Spent**≠**SpentA** event paying the payee. Payload = issuer Party (public) + payee Party + the **Receipt** witness + payee **vd_proof** + mint and spend anchors. A1 differs only by the nullifier tag (4243 vs 4242) and the **SpentA** event.

5.4 note-a2 – Note from a private issuer

The hard case: both parties are encrypted-Identity. The issuer hid behind **E_iss-for-rec** in the leaf’s **idHash**; the payee decrypts it under their own key and **verifiable_decrypt** reveals **M_iss**. Soundness – that **E_iss-for-rec** re-encrypts the issuer’s *registered* credential, addressed to *this* recipient – is now carried by the **issuer_reenc** binding the issuer ships with the note (the recipient-blinded re-encryption proof verified by **IdentityRegistry.verifyIssuerReenc**) plus the disclosed un-blinding scalar γ . **verify_receipt** composes the binding with the payee’s verifiable decryption and requires $M_{iss}^{reg} = C_i - \hat{T} + \gamma G = M_{named} = \text{payer.M}$, which algebraically forces

$pk_{rec} = Q$'s key – the *coupling* that proves the note was addressed to the payee – so the slip reads VALID and names the private issuer.

Payload = both Parties (private) + `vd_issuer` + the `issuer_reenc` binding + γ + payee self `vd_proof` + mint and spend anchors. The chain saw only the *blinded* binding ($\hat{T}$ hides M_{iss} , Q hides pk_{rec}); γ is disclosed only here, in the compelled receipt, where naming the issuer is the point (see *Data Revealed Per Step* in the Decryptability companion). A note whose issuer supplied *no* binding still prints, with `M_iss` recovered but stamped **UNVERIFIED ISSUER** (the payer leg is not provably the registered Identity); the payee and txn legs still verify.

6 The Physical Artifact (ESC/POS, 80mm, 203dpi, 576px)

576 dots wide; Font A (12x24) gives **48 columns**, Font B (9x17) gives 64. The body prints at 48 cols; the base64url payload prints at Font B / 64 cols to cut its line count. Output is 7-bit ASCII only (names transliterated if needed – see *Open Questions*). ESC/POS control: ESC @ init, ESC a n align, ESC E n emphasis, GS V cut; no graphics.

The human-readable parties and amount are *derived from the verified core* – a single source of truth – so what a person reads cannot drift from what a verifier checks. Only the NOTES section is free text.

```
=====
                ALBERTA  BUCK
              P A Y M E N T  R E C E I P T
=====
Receipt  ab1q7x...4f9          (sha256 handle)
Issued   2026-05-29 14:32 MST    AB-RCPT/1
-----
FROM (payer)                      [PRIVATE IDENTITY]
  Alice Johnson
  Alberta Identity Card - Alberta, Canada
  acct  0xa11ce...a11ce
  id pt 8f3a..1c    (point->name VERIFIED)
-----
TO (payee - you)                  [PRIVATE IDENTITY]
  Bob Smith
  Corporate Registration - Alberta, Canada
  acct  0x0b0b...0b0b
  id pt 2d77..9a    (point->name VERIFIED)
-----
TRANSACTION
  Type    EOA transfer (Identity-bound)
  Amount  $   250.00  BUCK
  When    2026-05-29 14:30 MST
  Chain 1  Block 1234567  Log #3
  Tx      0x9c2f...e1
-----
NOTES (informational - NOT verified)
  Invoice INV-2026-0042
  Consulting services, May 2026
```

```

-----
VERIFICATION          status @ issue: VALID
Re-verify: scan the payload below into
  albertabuck verify -
  (offline checks proofs + names; add --rpc
  to anchor to chain)

```

```

AB-RCPT/1.
o7VkdHlwZWVvYS1wcml2ZGNoYWluAWhjb250cmFjdH0j
aHJlZ2lzdHJ5VAr..Q3R4ZWVidWNrV... (wrapped,
Font B, ~30-50 lines depending on kind)
.END

```

```

=====
a record, not a bearer instrument
github.com/pjkundert/alberta-buck
=====

```

7 Verification Procedure

albertabuck verify [-|FILE] [--rpc URL]:

1. **Parse.** Strip to AB-RCPT/1.END, base64url-decode, CBOR-decode to the core map; reject unknown `v / type`.
2. **Tier 1 (always).** For `payer` and `payee`: recompute $M' = \text{keccak}(\text{identity}) \cdot G$ and require $M' = M$. Dispatch on `type` to the matching verifier over the embedded inputs: `approve_receipt_verify` (eoa-priv), `receipt_verify` (note-b1/a1), `verifiable_decrypt` for the payee self-naming (all) and the A2 issuer leg, and for `note-a2` also `verifyIssuerReenc` over the binding plus the coupling check $C_i - \hat{T} + \gamma G = \text{payer.M}$; eoa-pub needs no payer proof. Require the named `M`, `value`, and addresses to match the human-readable fields the renderer derived.
3. **Tier 2 (--rpc).** For each named address read `_pk`, `_E_addr`, `isPublicIdentity` and assert equality with the payload; read the log at (`txhash`, `logIndex`) and assert the `event`, `value` $\neq$ `face`, `recipient` $\neq$ `from` $\neq$ `to`, and (for Notes) `nullifier` and the `mint` match; assert the block is mined and final.
4. **Report.** VALID (+ named identities, amount) / INVALID (`reason`) / UNVERIFIED ISSUER for an A2 slip whose issuer supplied no binding.

8 Wallet UX and Storage

The receipt is generated from a selected received payment:

1. **Select** a row in tx history where the owner is `to` (a Transfer) or `recipient` (a Spent*).
2. **Gather** per the scenario table: a registry read (public payer), stored approve material (private EOA), or the stored note opening + mint reference (Notes); plus each party's `canonical_identity_data` from wallet storage.

3. **Self-name:** `verifiable_decrypt` the owner's own `E_addr`.
4. **Anchor:** read the `Transfer` / `Spent*` event and registry records.
5. **Prove + assemble:** generate the `vd_proof=(s)` with `=sk_owner`, build the core, *verify it locally*, set the status banner.
6. **Render + print:** ESC/POS at the configured width; or emit plain text.

Storage the wallet must retain (beyond keys): the owner's (`sk`, `identity_data`); per approved counterparty (`addr`, `identity_data`, `E_for_me`, `cp_proof`); per received Note the opening and its mint reference (`mint_txhash`, `cms` or `cm index`, `issuer_addr`, `issuer_sig`); and an event source (indexer or RPC) for the `Transfer` / `Spent*` logs and registry reads.

CLI surface (proposed). `albertabuck receipt <txref> [--width 48|64] [--escpos|--text]` to produce a slip; `albertabuck verify [-|FILE] [--rpc]` to check a scanned payload. Both reuse the wallet library; `verify` has no wallet-secret dependency (a third party runs it).

9 Security Considerations

- **PII on paper.** The slip is a deliberate disclosure: anyone holding it can name both parties. The display subset (`name` / `id_type` / `jurisdiction`) keeps the *printed* face modest, but the payload carries each party's **full canonical_identity_data** in base64 – base64 is *encoding*, *not encryption*. A receipt must be handled like a signed KYC disclosure, not like cash.
- **No value, no replay.** The slip redeems nothing; a referenced Note nullifier is already burned, an EOA transfer already mined. Re-printing or copying a slip moves no funds.
- **Generator honesty bounds.** Tier-1/2 bind the parties, amount, and event; the free-text NOTES are unbound, so a generator may write anything there – hence they are labelled *informational*, *not verified*, and excluded from the receipt id.
- **A2 issuer privacy.** The A2 binding is *blinded*: the chain sees $\hat{T} = r'pk_{rec} + \gamma G$ and $Q = pk_{rec} + \beta H$, revealing neither the issuer's M_{iss} nor the recipient's pk_{rec} . An earlier draft published the unblinded $T = r'pk_{rec}$, which leaks $M_{iss} = C_i - T$ – and `msg.sender` at mint *is* the issuer – so the blind is load-bearing. The receipt discloses γ (un-blinding M_{iss}) only here, under compulsion; a slip whose issuer shipped no binding still names the *recovered* point but is stamped UNVERIFIED ISSUER (not provably the *registered* Identity). Full collusion-resistant enforcement (every A2 leaf must carry a binding) awaits the mint-SNARK `issuerMode` signal – see the Decryptability companion.
- **Trust roots.** Soundness rests on the proof algebra (tier 1), the on-chain registry + event (tier 2), and the KYC issuer's $m \leftrightarrow \text{person}$ binding (out of scope; assumed via the PS credential).

10 Phasing

1. **Phase A (shipped).** Envelope + `verify` CLI + the plain-text renderer for all five kinds. `eoapub`, `eoapriv`, `noteb1`, `notea1` are soundly bound; the receipt **core** (`envelope.py` / `build_receipt.py` / `verify_receipt.py` / `render.py`) is the shipped wallet surface.

2. **Phase B (shipped, off-chain).** `note-a2` is now soundly bound: the `issuer_reenc` binding + γ + the coupling in `verify_receipt` name the private issuer's *registered* Identity; the slip reads **VALID**. What remains is on-chain *completeness* enforcement (the mint-SNARK `issuerMode` gate), so that a binding-less A2 leaf cannot be minted at all – the **UNVERIFIED ISSUER** path closes only then.
3. **Later.** The ESC/POS byte driver (Phase A shipped the plain-text driver; a port-9100 thermal driver is a new `Driver` subclass); optional QR rendering; payer-side ("I paid X") receipts; on-chain `RcptVerify` if a contract ever needs to gate on a receipt.

11 Open Questions

1. **Encoding for OCR.** `base64url` is compact but OCR-hostile (l/1/I, O/0); RFC 4648 `base32` is ~20% larger but robust to hand entry and OCR. Default `base64url`, or `base32` for paper? (The crypto payload is high-entropy, so compression buys little.)
2. **Unicode names.** Thermal output is ASCII; non-Latin `given_name` / `family_name` need transliteration for the *printed* face, while the payload keeps the exact UTF-8 `canonical_identity_data` (the hash preimage). How to transliterate without implying the printed name is the canonical one?
3. **Point-only fallback.** When the wallet lacks a private counterparty's `identity_data` preimage, should it print the *M* fingerprint + issuer reference (de-anonymizable by the issuer under subpoena), or refuse?
4. **Payload length.** Offline-complete + text-only makes Notes / private-EOA payloads ~2-2.7 KB of `base64` (~30-50 Font-B lines). Acceptable for thermal, or offer a `--refs-only` compact mode that requires `--rpc` to verify?
5. **Time and locale.** Print block timestamp (consensus truth) or wall clock, and in UTC or the wallet's locale? The core stores unix block time; the printed string is a render choice.